

Production Planning & Tracking System

Comprehensive Technical Architecture, Workflow, and Implementation Report

Assigned to: Vaishnavi | Project 2 | Technologies: Java (Micronaut), Vue.js 3, MySQL 8.0, ConcurrentHashMap

1. Executive Summary & Objective

In manufacturing enterprises—ranging from automotive assembly plants to precision machining facilities—efficient operations depend on the ability to schedule target production volumes and rigorously compare them with actual daily shop-floor output. Without automated tracking, manufacturers suffer from delayed deliveries, inventory imbalances, line bottlenecks, and hidden production deficits.

The primary objective of the Production Planning & Tracking System is to bridge this operational gap by offering an intuitive, end-to-end digital tracking platform. The system empowers production managers, line engineers, and plant supervisors to define product master catalogues, establish monthly production target plans, log actual shift-by-shift manufacturing output, and automatically calculate real-time output variance and target achievement percentages.

2. Technology Stack & Architectural Decisions

The system has been architected using a modern, loosely-coupled, two-tier client-server architecture comprising a high-performance Java backend REST service, an interactive Vue.js 3 Single Page Application (SPA), and a resilient dual-layer data repository combining MySQL 8.0 with ConcurrentHashMap caching.

2.1 Frontend Technologies

- **Vue.js 3 (Composition API):** Modern JavaScript framework utilizing '<script setup>' syntax for reactive state management, modular component design, and lightweight rendering.
- **Vite 5:** Blazing-fast build tool and development server providing instant Hot Module Replacement (HMR) and an integrated reverse proxy that eliminates Cross-Origin Resource Sharing (CORS) friction.
- **Vue Router 4:** Client-side routing engine managing seamless transitions across the 5 core application views with navigation authentication guards.
- **Modern Clean CSS Design System:** Custom styling with CSS variables, responsive grid layouts, card abstractions, status indicators, and modal window animations.

2.2 Backend Technologies

- **Java 25 (Temurin / OpenJDK):** Modern Long-Term Support Java runtime utilizing modern language capabilities and optimized garbage collection.
- **Micronaut Framework 5:** A modern, lightweight full-stack JVM framework. Unlike legacy frameworks that rely on heavyweight runtime reflection and dynamic class generation, Micronaut computes dependency injection and route mapping at compile time (Ahead-of-Time). This results in sub-second startup (~600ms) and minimal memory footprint.

- **Micronaut Netty Server:** High-throughput, asynchronous, non-blocking I/O HTTP web server handling concurrent REST API requests.
- **Micronaut Serde (Jackson):** Reflection-free, compile-time generated JSON serialization and deserialization annotated via '@Serdeable'.

2.3 Database & Concurrency Layer

- **MySQL Server 8.0:** Industry-standard relational database management system ('production_db') providing ACID-compliant persistent storage across relational tables.
- **Java Database Connectivity (JDBC):** Clean, standard SQL queries executed via 'java.sql.PreparedStatement' and parameterized execution, ensuring zero SQL injection vulnerabilities and maximum transparency.
- **ConcurrentHashMap:** High-performance, thread-safe in-memory cache implemented within the repository layer. Uses bucket-level lock striping and non-blocking reads to provide instant O(1) retrieval of entities and prevent race conditions when multiple operators submit production entries simultaneously.

3. Mathematical Business Logic & Calculation Formulas

The core analytical intelligence of the system lies in its automated variance calculation and performance categorization. These metrics provide instantaneous feedback to plant management on whether manufacturing lines are ahead, on track, or lagging behind scheduled quotas.

3.1 Difference (Variance) Calculation

The production variance indicates the absolute volume gap between actual manufactured units and the target schedule:

$$\text{Difference} = \text{Produced Quantity} - \text{Planned Quantity}$$

- **Positive Difference (> 0):** Surplus output. The production line exceeded its quota (Ahead of target).
- **Zero Difference (= 0):** Exact achievement. Actual output matches the planned target perfectly.
- **Negative Difference (< 0):** Production deficit. The line experienced a shortfall and is lagging behind target.

3.2 Target Achievement Percentage Calculation

The achievement percentage normalizes the production progress into a standardized performance indicator:

$$\text{Achievement \%} = (\text{Produced Quantity} / \text{Planned Quantity}) * 100$$

To ensure mathematical safety, the system checks for division-by-zero scenarios: if Planned Quantity is zero, the achievement percentage safely defaults to 100% (if units were produced) or 0% (if no units were produced).

3.3 Performance Status Categorization Thresholds

- **ACHIEVED (>= 100.0%):** Target fulfilled or exceeded. Displayed with a vibrant Green badge and filled progress bar.
- **ON_TRACK (80.0% to 99.9%):** Production is within acceptable operational tolerance. Displayed with a Yellow / Blue badge.

- **LAGGING (< 80.0%)**: Critical production deficit requiring supervisory intervention. Displayed with a Red badge.

4. End-to-End System Workflow & Project Flow

The operational lifecycle of the Production Planning & Tracking System follows a clear 5-stage sequential workflow:

Stage 1: User Authentication Flow

The user accesses the portal at 'http://localhost:5173/login'. The operator or plant manager enters their credentials ('admin' / 'admin123'). The Vue.js frontend sends a POST request to '/api/auth/login'. Upon validation by the Micronaut AuthController, a session token and user profile payload are returned and stored in browser 'localStorage'. The client router automatically redirects the authenticated user to the executive dashboard.

Stage 2: Product Master Catalogue Setup Flow

The plant manager navigates to the 'Product Master' screen ('/products'). The system retrieves the existing parts catalogue from MySQL. When a new part is introduced (e.g. 'PRD-005: Turbocharger Core'), the user clicks '+ Add New Product', enters the part code, description, category, and unit of measure ('Units', 'Sets', 'Kg'). The frontend submits the data to 'POST /api/products', which executes an INSERT statement into MySQL and registers the item in the 'ConcurrentHashMap' cache.

Stage 3: Production Target Planning Flow

Production targets are established in the 'Production Planning' screen ('/planning'). The manager selects a product from the dropdown, chooses the target production month (e.g. '2026-09'), and enters the planned target volume (e.g. 500 units). Submitting the form calls 'POST /api/plans', inserting the planned quota into MySQL and updating total scheduled plant capacity.

Stage 4: Daily Shop-Floor Production Logging Flow

At the conclusion of each manufacturing shift, line supervisors access the 'Daily Entry' screen ('/production'). They select the product, log the calendar date, choose the shift ('Morning', 'Evening', or 'Night'), and record actual manufactured output (e.g. 180 units) along with operator remarks. Submitting the entry triggers 'POST /api/production', writing a new log entry into the MySQL 'production_entries' table.

Stage 5: Automated Variance Analysis & Summary Reporting Flow

Management reviews plant performance in the 'Variance Report' screen ('/reports'). When the page loads, the frontend calls 'GET /api/reports/variance'. The backend calculates the sum of all planned targets and the sum of all actual daily production per product. It computes the Difference and Achievement %, assigns the status badge ('Achieved', 'On Track', 'Lagging'), and returns the aggregated dataset. The frontend displays executive KPI cards, an interactive variance table with search/filters, CSV export, and print capabilities.

5. How the Frontend and Backend Communicate

Communication between the Vue.js Single Page Application and the Micronaut Java backend is entirely decoupled, stateless, and based on standard RESTful HTTP protocols transmitting JSON payloads.

5.1 The Request-Response Cycle & Reverse Proxy

During local development, browser security policies restrict cross-origin network calls between different ports ('http://localhost:5173' vs. 'http://localhost:8080'). To solve this cleanly without CORS errors, Vite is configured with a built-in reverse proxy in 'vite.config.js':

```
server: {
  port: 5173,
  proxy: {
    '/api': {
      target: 'http://localhost:8080',
      changeOrigin: true
    }
  }
}
```

When the Vue frontend executes 'fetch('/api/products')', Vite intercepts the request on port 5173 and transparently proxies it to the Micronaut server on port 8080. The browser perceives the request as same-origin, completely eliminating CORS blockage.

5.2 Centralized API Client Service ('api.js')

All HTTP requests are encapsulated inside a modular service module located at 'frontend/src/services/api.js'. A reusable helper function handles headers, JSON stringification, HTTP status verification, and error interception:

```
async function request(endpoint, options = {}) {
  const config = {
    headers: { 'Content-Type': 'application/json' },
    ...options
  };
  const response = await fetch('/api' + endpoint, config);
  if (!response.ok) throw new Error('API Error ' + response.status);
  return await response.json();
}
```

5.3 REST API Endpoints Specification Table

The backend exposes clean, declarative REST endpoints documented below:

- **POST /api/auth/login:** Authenticates operator credentials. Request: {username, password}. Response: {success, token, name, role}.
- **GET /api/products:** Retrieves full catalogue of products from MySQL and ConcurrentHashMap cache.
- **POST /api/products:** Creates new product. Request: {productCode, productName, category, unitOfMeasure, description}.
- **DELETE /api/products/{id}:** Removes a product record from database and cache.
- **GET /api/plans:** Retrieves all production targets scheduled across months.
- **POST /api/plans:** Establishes a new target plan. Request: {productId, planMonth, plannedQuantity, notes}.

- **GET /api/production:** Retrieves historical log of daily shift output entries.
- **POST /api/production:** Records actual manufactured output for a shift. Request: {productId, entryDate, producedQuantity, shift, remarks}.
- **GET /api/reports/variance:** Generates real-time variance analysis combining planned vs. actual output, returning Difference, Achievement %, and Status.

6. Database Schema & Dual-Layer Architecture

The system implements a resilient dual-layer data architecture where MySQL provides ACID persistence and ConcurrentHashMap provides lock-free, in-memory concurrency:

6.1 MySQL Relational Tables Structure

- **Table 'products':** Columns: id (BIGINT, Primary Key, Auto Increment), product_code (VARCHAR 50, Unique), product_name (VARCHAR 255), category (VARCHAR 100), unit_of_measure (VARCHAR 50), description (TEXT).
- **Table 'production_plans':** Columns: id (BIGINT, Primary Key, Auto Increment), product_id (BIGINT, Foreign Reference), product_name (VARCHAR 255), plan_month (VARCHAR 20), planned_quantity (DOUBLE), notes (TEXT).
- **Table 'production_entries':** Columns: id (BIGINT, Primary Key, Auto Increment), product_id (BIGINT, Foreign Reference), product_name (VARCHAR 255), entry_date (VARCHAR 20), produced_quantity (DOUBLE), shift (VARCHAR 50: Morning/Evening/Night), remarks (TEXT).

6.2 Role of ConcurrentHashMap in Concurrency

Standard Java HashMaps are not thread-safe: concurrent reads and writes can lead to infinite loops, hash collision corruption, or 'ConcurrentModificationException'. 'ConcurrentHashMap' divides its internal storage into segmented buckets and employs compare-and-swap (CAS) lock striping. Multiple threads can read concurrently without locking, while writes lock only the specific bucket being modified. This ensures the system remains responsive under heavy concurrent loads.

7. Viva & Technical Interview Questions with Model Answers

Q1: What is the primary objective of this project?

Answer: The objective is to track planned production quotas versus actual shop-floor manufactured quantities, calculate variance (Difference) and target Achievement %, and present actionable reports to plant managers to prevent delivery delays.

Q2: Why did you choose Micronaut over standard Spring Boot?

Answer: Micronaut performs dependency injection and annotation validation at compile-time rather than runtime. This eliminates runtime reflection, reduces memory consumption significantly, and enables the application to start in under 1 second (~639ms), making it ideal for microservices and responsive APIs.

Q3: How is the production variance calculated, and what do negative values mean?

Answer: Variance is calculated as 'Produced Quantity - Planned Quantity'. A negative difference represents a production deficit (shortfall), meaning the production line is lagging behind schedule and needs corrective action.

Q4: How do the frontend and backend communicate without CORS issues?

Answer: The frontend communicates via asynchronous HTTP fetch calls. During development, Vite's reverse proxy intercepts all calls to '/api/*' on port 5173 and forwards them to the Micronaut backend on port 8080, preventing browser CORS origin blocking.

Q5: How does your architecture handle multi-threading and database persistence?

Answer: We use a dual-layer architecture: MySQL 8.0 serves as the persistent relational database, while Java's 'ConcurrentHashMap' operates in the repository layer as a thread-safe cache with bucket-level lock striping to deliver instant $O(1)$ lookups and thread safety.